VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT
on

Machine Learning (23CS6PCMAL)

Submitted by

G M Kusuma(1BM24CS405)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Sep-2024 to Jan-2025

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "Machine Learning (23CS6PCMAL)" carried out by **G M Kusuma (1BM24CS405),** who is bonafide student of **B.M.S. College of Engineering.** It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Machine Learning (23CS6PCMAL) work prescribed for the said degree.

Lab faculty Incharge Name Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE

Index

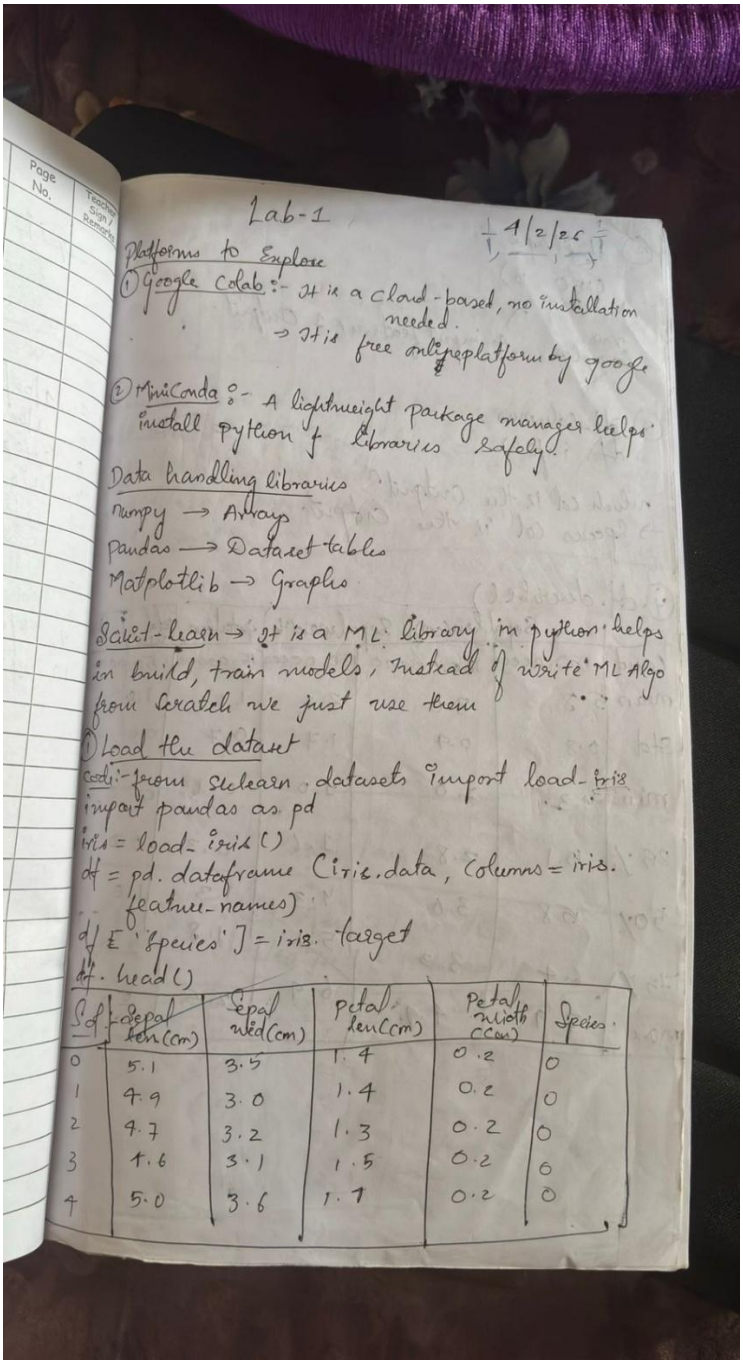
Sl. No.	Date	Experiment Title	Page No.
1	21-2-2025	Write a python program to import and export data using Pandas library functions	
2	3-3-2025	Demonstrate various data pre-processing techniques for a given dataset	
3	10-3-2025	Implement Linear and Multi-Linear Regression algorithm using appropriate dataset	
4	17-3-2025	Build Logistic Regression Model for a given dataset	
5	24-3-2025	Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.	
6	7-4-2025	Build KNN Classification model for a given dataset.	
7	21-4-2025	Build Support vector machine model for a given dataset	
8	5-5-2025	Implement Random forest ensemble method on a given dataset.	
9	5-5-2025	Implement Boosting ensemble method on a given dataset.	
10	12-5-2025	Build k-Means algorithm to cluster a set of data stored in a .CSV file.	
11	12-5-2025	Implement Dimensionality reduction using Principal Component Analysis (PCA) method.	

Github Link: https: https://github.com/Kusumagm07/ML-LAB

Program 1

Write a python program to import and export data using Pandas library functions

Screenshot



Lab-1

4/2/26

Platforms to Explore

① Google Colab :- It is a cloud-based, no installation needed.
→ It is free online platform by google.

② MiniConda :- A lightweight package manager helps install python & libraries safely.

Data handling libraries

numpy → Array
pandas → Dataset tables
Matplotlib → Graphs

Scikit-learn → It is a ML library in python helps in build, train models, instead of write ML Algo from scratch we just use them

① Load the dataset

code:-
```
from sklearn.datasets import load_iris
import pandas as pd
iris = load_iris()
df = pd.dataframe(iris.data, columns = iris.feature_names)
df['species'] = iris.target
df.head()
```

	sepal len(cm)	sepal wid(cm)	petal len(cm)	Petal width (cm)	Species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.7	0.2	0

② df. shape

(150 , 5)

↑ 150 samples ↑ 4 input features + 1 Output.

③. df. columns

df. info()

which col is the Output?

→ Species col is the Output.

④. df. describe()

	Sepal len(cm)	Sepal width	petal len	petal wid
count	150.00000	150.00000	150.00000	150.00
mean	5.8	3.0	3.7	1.1
std	0.8	0.9	1.7	0.7
min	4.3	2.0	1.0	0.1
25%	5.1	2.8	1.6	0.3
50%	5.8	3.0	4.3	1.3
75%	6.4	3.3	5.1	1.8
max	7.9	4.4	6.9	2.5

Output.

df['species'].value_counts()

Species	count
0	50
1	50
2	50

Yes, the classes are balanced.

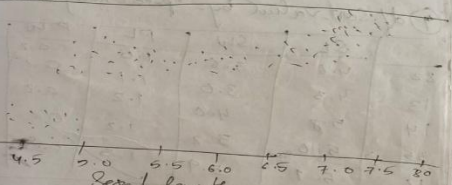
import matplotlib.pyplot as plt

plt.scatter(df['sepal len(cm)'], df['petal len(cm)'])

plt.xlabel("sepal len")

plt.ylabel("petal len")

plt.show()



df[df['species'] == 0].head()

df[df['species'] == 1].head()

	Sep len	Sepal wid	petal len	petal wid	species
50	7.0	3.2	4.7	1.4	1
51	6.4	3.2	4.5	1.5	1
52	6.9	3.1	4.9	1.5	1
53	5.5	2.3	4.0	1.3	1
54	6.5	2.8	4.6	1.5	1

Code:

```python
from sklearn.datasets import load_iris
import pandas as pd
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target
df.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

```python
df.shape
```

```
(150, 5)
```

```python
df.columns
```

```
Index(['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
       'petal width (cm)', 'species'],
      dtype='object')
```

```python
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
 4   species            150 non-null    int64
dtypes: float64(4), int64(1)
memory usage: 6.0 KB
```

```python
df.describe()
```

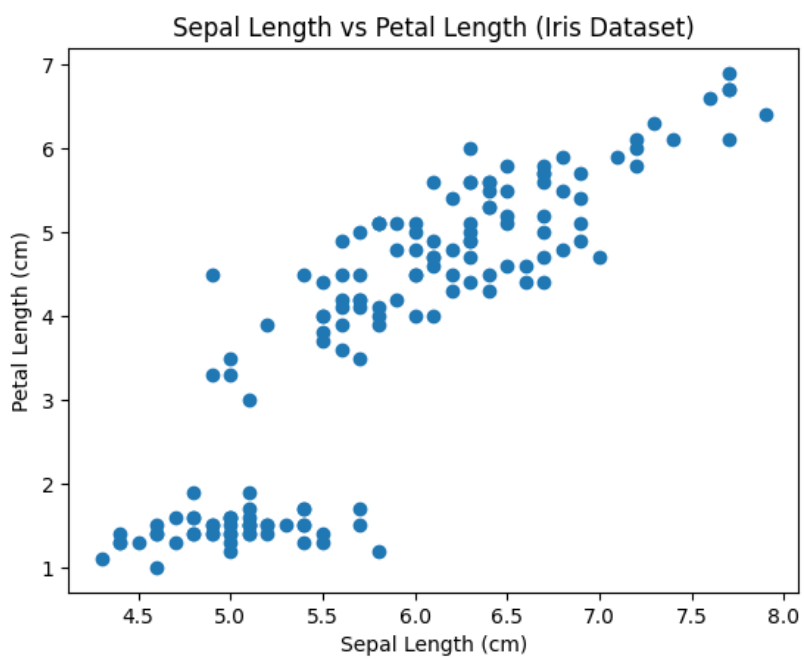
	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333	1.000000
std	0.828066	0.435866	1.765298	0.762238	0.819232
min	4.300000	2.000000	1.000000	0.100000	0.000000
25%	5.100000	2.800000	1.600000	0.300000	0.000000
50%	5.800000	3.000000	4.350000	1.300000	1.000000
75%	6.400000	3.300000	5.100000	1.800000	2.000000
max	7.900000	4.400000	6.900000	2.500000	2.000000

```python
df['species'].value_counts()
```

species	count
0	50
1	50
2	50

dtype: int64

```python
# Import required libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

# Load the Iris dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target

# Scatter plot: Sepal Length vs Petal Length
plt.scatter(df['sepal length (cm)'], df['petal length (cm)'])
plt.xlabel("Sepal Length (cm)")
plt.ylabel("Petal Length (cm)")
plt.title("Sepal Length vs Petal Length (Iris Dataset)")
plt.show()
```



```
df[df['species'] == 0].head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

```
df[df['species'] == 1].head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
50	7.0	3.2	4.7	1.4	1
51	6.4	3.2	4.5	1.5	1
52	6.9	3.1	4.9	1.5	1
53	5.5	2.3	4.0	1.3	1
54	6.5	2.8	4.6	1.5	1

```
df.corr()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
sepal length (cm)	1.000000	-0.117570	0.871754	0.817941	0.782561
sepal width (cm)	-0.117570	1.000000	-0.428440	-0.366126	-0.426658
petal length (cm)	0.871754	-0.428440	1.000000	0.962865	0.949035
petal width (cm)	0.817941	-0.366126	0.962865	1.000000	0.956547
species	0.782561	-0.426658	0.949035	0.956547	1.000000

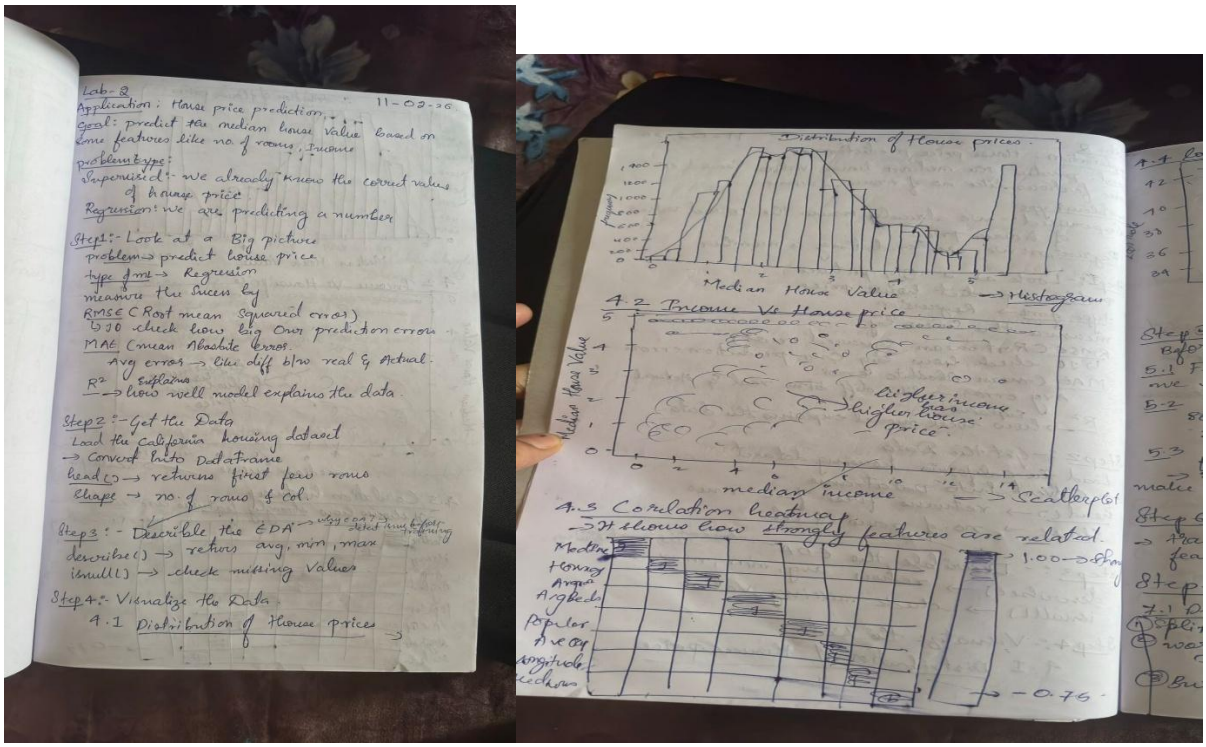
```
df.sort_values(by='petal length (cm)').head()
```

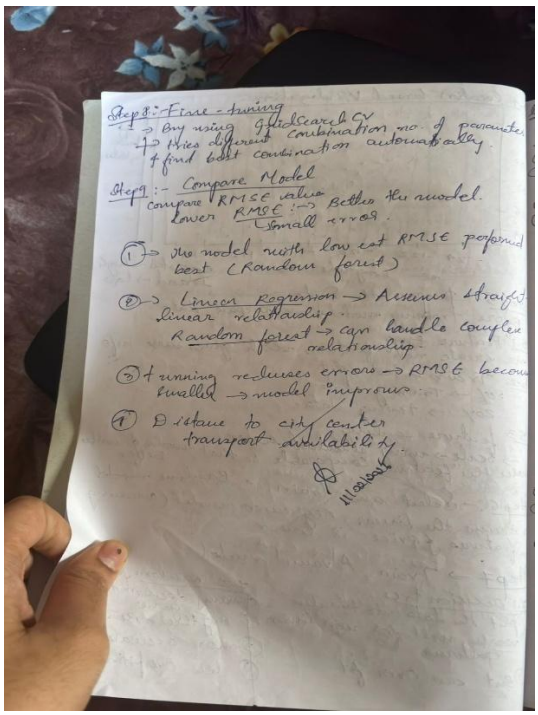
	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
22	4.6	3.6	1.0	0.2	0
13	4.3	3.0	1.1	0.1	0
14	5.8	4.0	1.2	0.2	0
35	5.0	3.2	1.2	0.2	0
16	5.4	3.9	1.3	0.4	0

Program 2

Demonstrate various data pre-processing techniques for a given dataset

Screenshot





Code:

```
from sklearn.datasets import load_iris
import pandas as pd
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target
df.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

```
df.shape
```

```
(150, 5)
```

```
df.columns
```

```
Index(['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
       'petal width (cm)', 'species'],
      dtype='object')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
 4   species            150 non-null    int64
dtypes: float64(4), int64(1)
memory usage: 6.0 KB
```

```
df.describe()
```

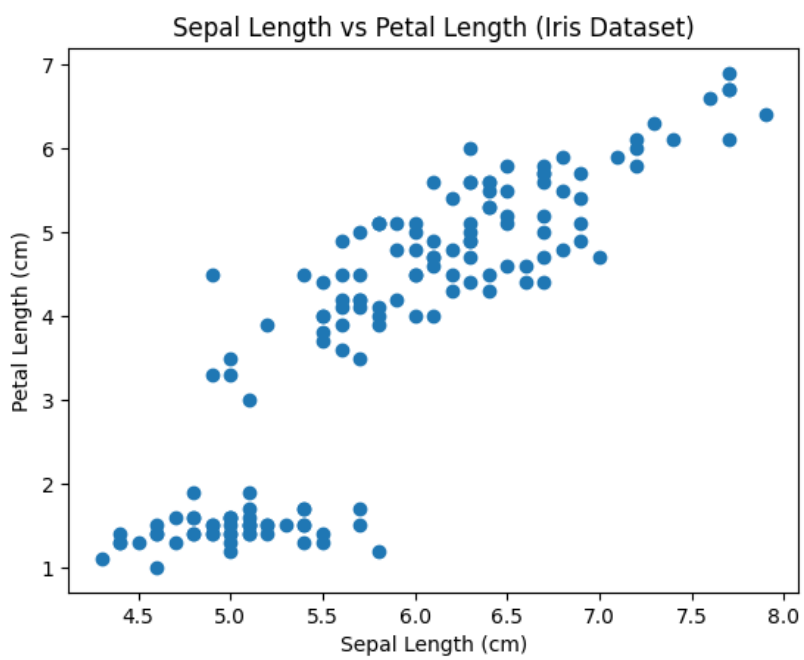
	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333	1.000000
std	0.828066	0.435866	1.765298	0.762238	0.819232
min	4.300000	2.000000	1.000000	0.100000	0.000000
25%	5.100000	2.800000	1.600000	0.300000	0.000000
50%	5.800000	3.000000	4.350000	1.300000	1.000000
75%	6.400000	3.300000	5.100000	1.800000	2.000000
max	7.900000	4.400000	6.900000	2.500000	2.000000

```python
df['species'].value_counts()
```

species	count
0	50
1	50
2	50

dtype: int64

```python
# Import required libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

# Load the Iris dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target

# Scatter plot: Sepal Length vs Petal Length
plt.scatter(df['sepal length (cm)'], df['petal length (cm)'])
plt.xlabel("Sepal Length (cm)")
plt.ylabel("Petal Length (cm)")
plt.title("Sepal Length vs Petal Length (Iris Dataset)")
plt.show()
```



```
df[df['species'] == 0].head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

```
df[df['species'] == 1].head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
50	7.0	3.2	4.7	1.4	1
51	6.4	3.2	4.5	1.5	1
52	6.9	3.1	4.9	1.5	1
53	5.5	2.3	4.0	1.3	1
54	6.5	2.8	4.6	1.5	1

```
df.corr()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
sepal length (cm)	1.000000	-0.117570	0.871754	0.817941	0.782561
sepal width (cm)	-0.117570	1.000000	-0.428440	-0.366126	-0.426658
petal length (cm)	0.871754	-0.428440	1.000000	0.962865	0.949035
petal width (cm)	0.817941	-0.366126	0.962865	1.000000	0.956547
species	0.782561	-0.426658	0.949035	0.956547	1.000000

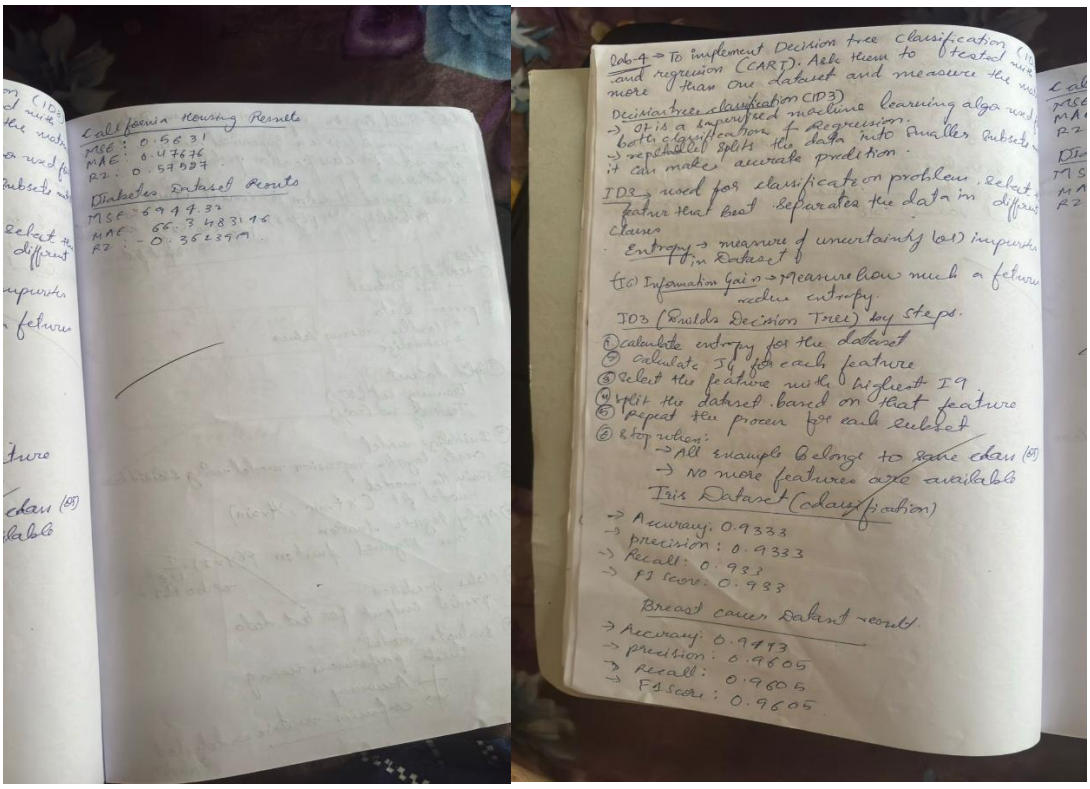
```
df.sort_values(by='petal length (cm)').head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
22	4.6	3.6	1.0	0.2	0
13	4.3	3.0	1.1	0.1	0
14	5.8	4.0	1.2	0.2	0
35	5.0	3.2	1.2	0.2	0
16	5.4	3.9	1.3	0.4	0

Program 3

Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.

Screenshot



Code:

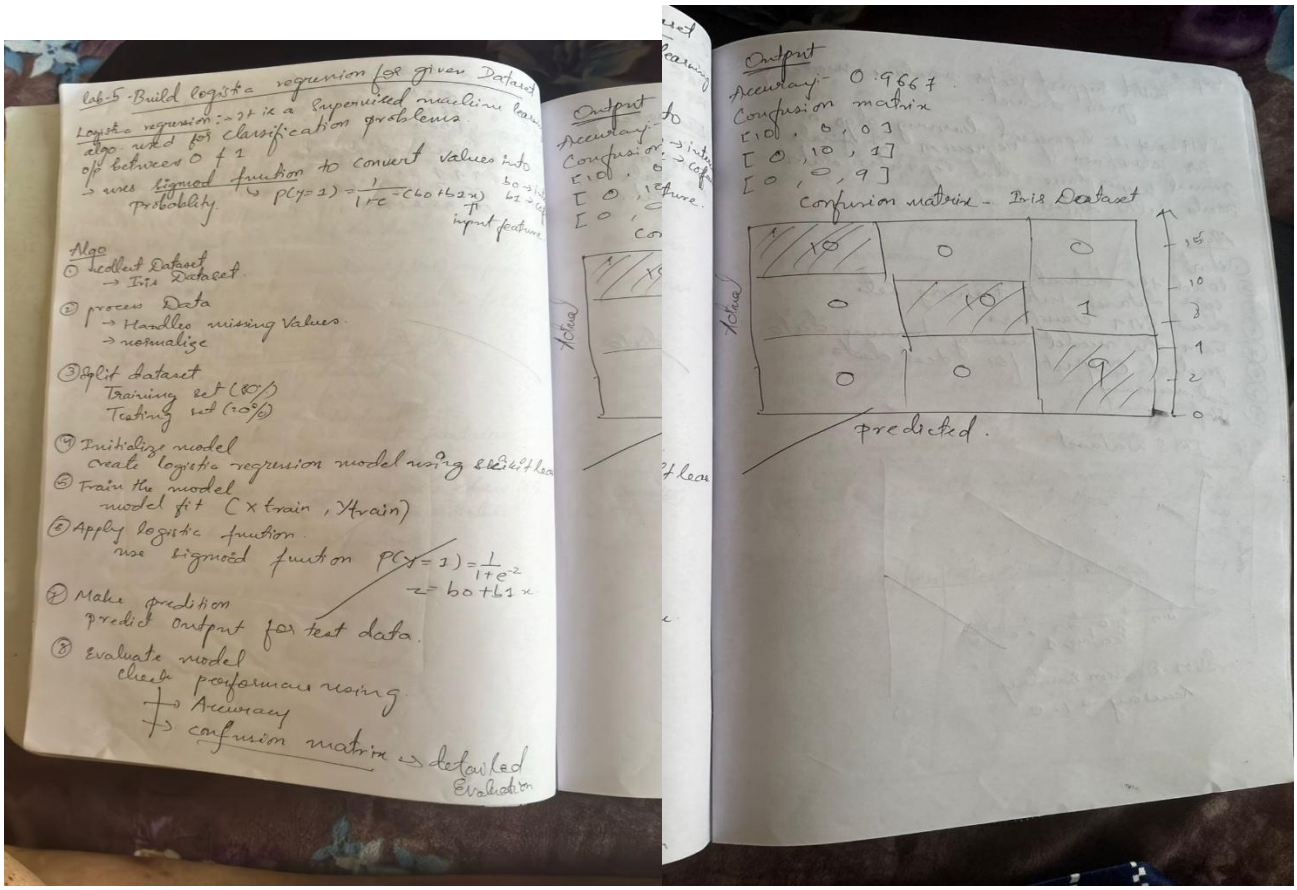
```python
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = DecisionTreeClassifier(criterion='entropy', random_state=42)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

new_sample = [[5.1, 3.5, 1.4, 0.2]]

prediction = model.predict(new_sample)

print("Predicted class:", prediction[0])
print("Predicted species:", iris.target_names[prediction][0])
```

```
Accuracy: 1.0
Predicted class: 0
Predicted species: setosa
```

Program 4

Implement Linear and Multi-Linear Regression algorithm using appropriate dataset

Screenshot



Code:

```
X = df.drop('MedHouseVal', axis=1)
y = df['MedHouseVal']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
lin_reg = LinearRegression()
lin_reg.fit(X_train_scaled, y_train)

y_pred_lr = lin_reg.predict(X_test_scaled)

rmse_lr = np.sqrt(mean_squared_error(y_test, y_pred_lr))
mae_lr = mean_absolute_error(y_test, y_pred_lr)
r2_lr = r2_score(y_test, y_pred_lr)

rmse_lr, mae_lr, r2_lr
```

```
(np.float64(0.7449540928043491), 0.5330139776645934, 0.5765012224420747)
```

```
dt = DecisionTreeRegressor(random_state=42)
dt.fit(X_train, y_train)

y_pred_dt = dt.predict(X_test)
rmse_dt = np.sqrt(mean_squared_error(y_test, y_pred_dt))

rmse_dt
```

```
np.float64(0.7138377373947808)
```

```
rf = RandomForestRegressor(random_state=42)
rf.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)
rmse_rf = np.sqrt(mean_squared_error(y_test, y_pred_rf))

rmse_rf
```

```
np.float64(0.5128644706054181)
```

```
param_grid = {
    'n_estimators': [50, 100],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5]
}

grid_search = GridSearchCV(
    rf,
    param_grid,
    cv=3,
    scoring='neg_mean_squared_error'
)
```

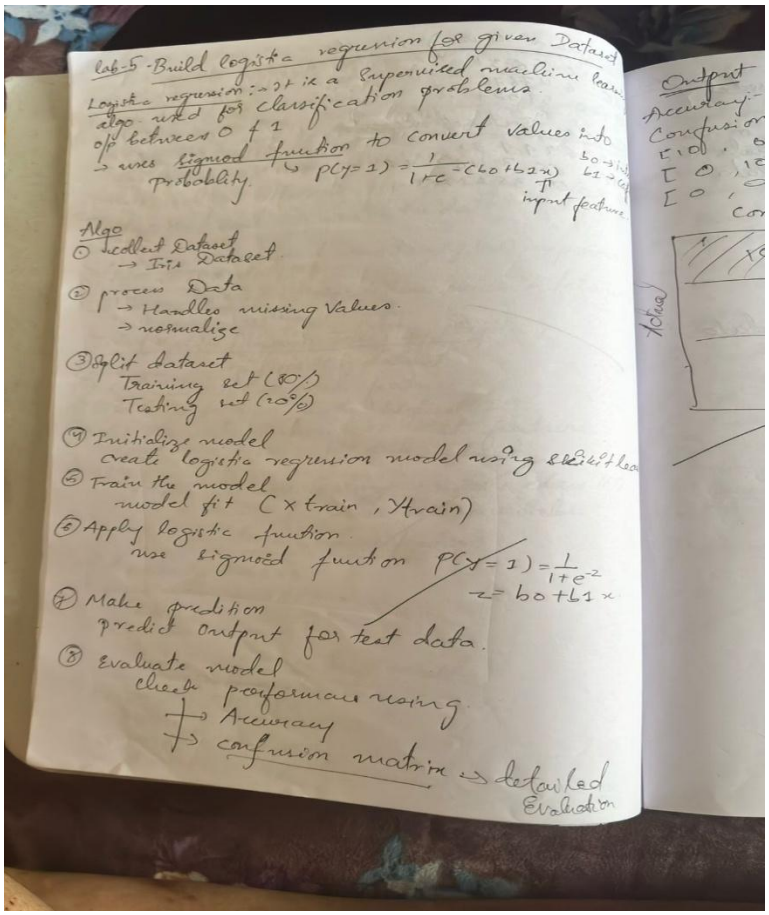
```python
results = pd.DataFrame({
    'Model': ['Linear Regression',
              'Decision Tree',
              'Random Forest',
              'Tuned Random Forest'],
    'RMSE': [rmse_lr, rmse_dt, rmse_rf, rmse_best]
})

results
```

	Model	RMSE
0	Linear Regression	0.744954
1	Decision Tree	0.713838
2	Random Forest	0.512864
3	Tuned Random Forest	0.513263

Program 5

Build Logistic Regression Model for a given dataset

Screenshot

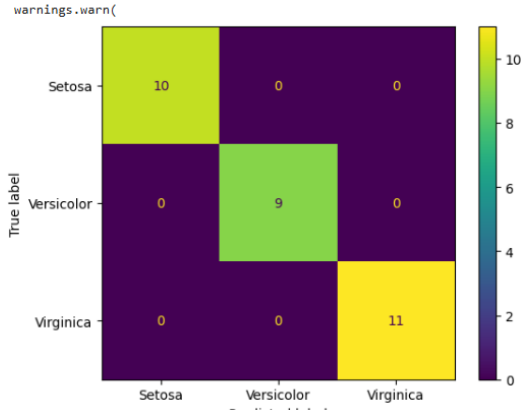


Code:

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn import metrics
import matplotlib.pyplot as plt

# Load correct file
iris = pd.read_csv("iris (1).csv")

print(iris.head())

# Features and target
X = iris.drop('species', axis='columns')
y = iris['species']

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Model
model = LogisticRegression(multi_class='multinomial', solver='lbfgs')
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```python
# Confusion Matrix
cm = metrics.confusion_matrix(y_test, y_pred)
disp = metrics.ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Setosa", "Versicolor", "Virginica"]
)
disp.plot()
plt.show()
```

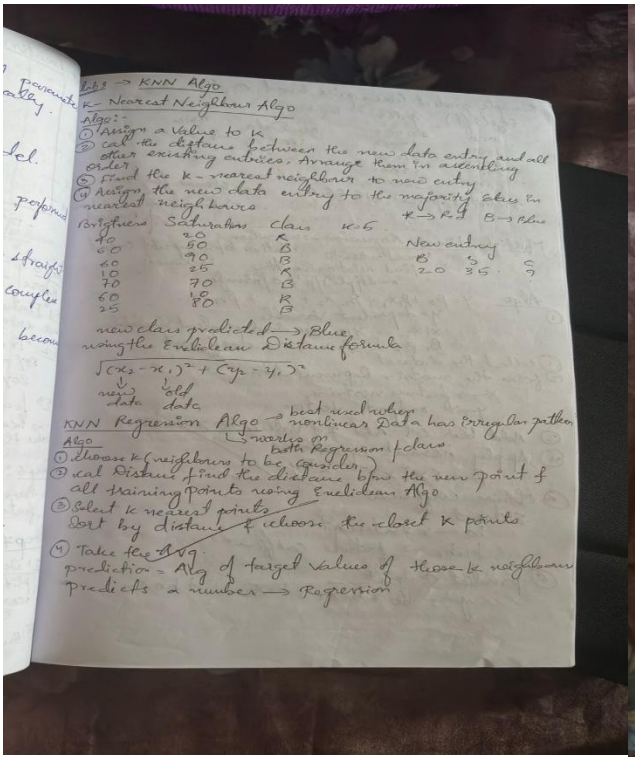
```
   sepal_length  sepal_width  petal_length  petal_width      species
0           5.1          3.5           1.4          0.2  Iris-setosa
1           4.9          3.0           1.4          0.2  Iris-setosa
2           4.7          3.2           1.3          0.2  Iris-setosa
3           4.6          3.1           1.5          0.2  Iris-setosa
4           5.0          3.6           1.4          0.2  Iris-setosa
Accuracy: 1.0
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated i
  warnings.warn(
```



Program 6

Build KNN Classification model for a given dataset

Screenshot



Lab 3 → KNN Algo
K- Nearest Neighbour Algo
Algo:
① Assign a value to K
② cal the distance between the new data entry and all other existing entries. Arrange them in ascending order
③ Find the K - nearest neighbors to new entry
④ Assign the new data entry to the majority class in nearest neighbours
K → Red B → Blue

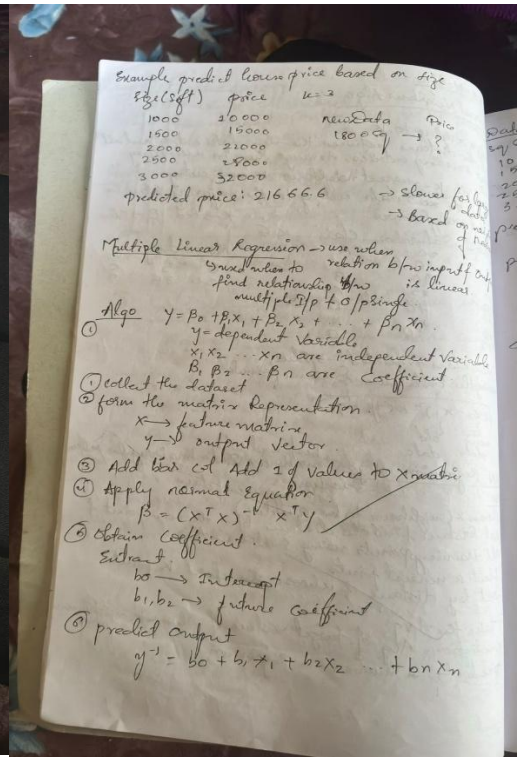
Brightness	Saturation	class	K=5
40	20	R	
60	50	B	
60	90	B	
10	25	R	
70	70	B	
60	10	R	
25	80	B	

New entry
B 4 5
20 35 ?

new class predicted → Blue
using the Euclidean Distance formula

$\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$
new data, old data

KNN Regression Algo → best used when nonlinear Data has irregular pattern
→ works on both Regression & class
Algo
① choose K (neighbours to be consider)
② cal Distance find the distance b/w the new point & all training points using Euclidean Algo
③ Select K nearest points
Sort by distance & choose the closest K points
④ Take the Avg
prediction = Avg of target values of those K neighbours
Predicts a number → Regression



Example predict house price based on size
K=3

size(sqft)	price
1000	10000
1500	15000
2000	21000
2500	25000
3000	32000

new data size 1800 sq → ?

predicted price: 21666.6 → closer for higher weight
→ based on weight of nearest

Multiple Linear Regression → use when used when to find relationship b/w multiple I/p & O/p single relation b/w input & output is linear

Algo
① $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$
y = dependent variable
$x_1 x_2 \dots x_n$ are independent variables
$\beta_1 \beta_2 \dots \beta_n$ are coefficient
① collect the dataset
② form the matrix Representation
X → feature matrix
y → output vector
③ Add bias col Add 1 of values to X matrix
④ Apply normal Equation
$\beta = (X^T X)^{-1} X^T y$
⑤ Obtain coefficient.
Extract
b_0 → Intercept
b_1, b_2 → future coefficient
⑥ predict output
$y^{-1} = b_0 + b_1 x_1 + b_2 x_2 \dots + b_n x_n$

Code:

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from sklearn.metrics import mean_squared_error, r2_score

from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import DecisionTreeRegressor
```

```
from sklearn.datasets import load_iris

data = load_iris()

X = data.data
y = data.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
model = DecisionTreeClassifier(criterion='entropy')

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

print(classification_report(y_test, y_pred))
```

```
Accuracy: 1.0
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       1.00      1.00      1.00        13
           2       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45
```

```
from sklearn.datasets import load_wine

data = load_wine()

X = data.data
y = data.target
```

```
from sklearn.datasets import load_diabetes

data = load_diabetes()

X = data.data
y = data.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
regressor = DecisionTreeRegressor()

regressor.fit(X_train, y_train)

y_pred = regressor.predict(X_test)
```

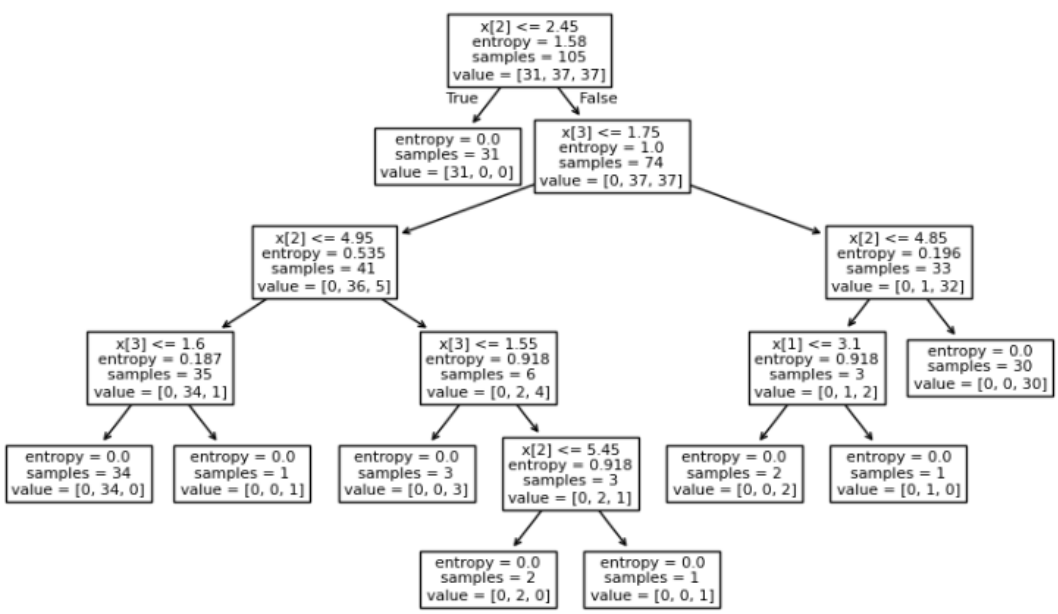
```
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error:", mse)
print("R2 Score:", r2)
```

```
Mean Squared Error: 5358.458646616541
R2 Score: 0.0073817790735694055
```

```
from sklearn.datasets import fetch_california_housing

data = fetch_california_housing()

X = data.data
y = data.target
```

```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))
plot_tree(model)
plt.show()
```

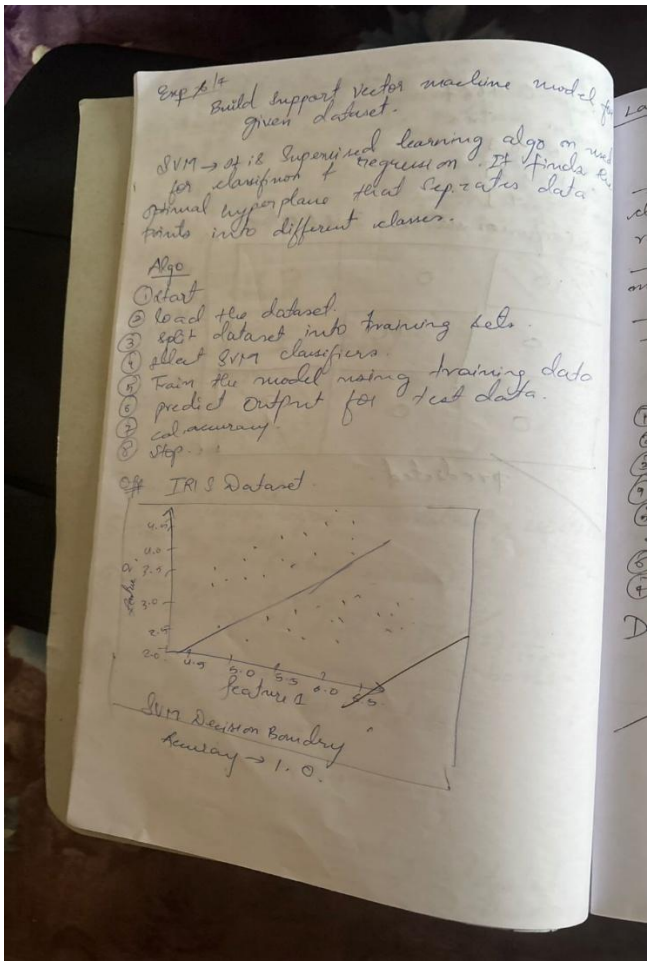
...



Program 7

Build Support vector machine model for a given dataset

Screenshot



Exp No 14
Build support vector machine model for given dataset.

SVM → it is supervised learning algo on used for classification & regression. It finds the optimal hyperplane that separates data points into different classes.

Algo
1. Start
2. load the dataset.
3. split dataset into training sets.
4. select SVM classifiers.
5. Train the model using training data
6. predict output for test data.
7. cal accuracy.
8. stop.

IRIS Dataset.

Accuracy → 1.0.

Code:

```python
# Import libraries
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.preprocessing import StandardScaler

# Load dataset
data = datasets.load_iris()
X = data.data
y = data.target

# Feature scaling (important for SVM)
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Split dataset (fixed for same results every time)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create SVM model
model = SVC(kernel='linear')

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
```

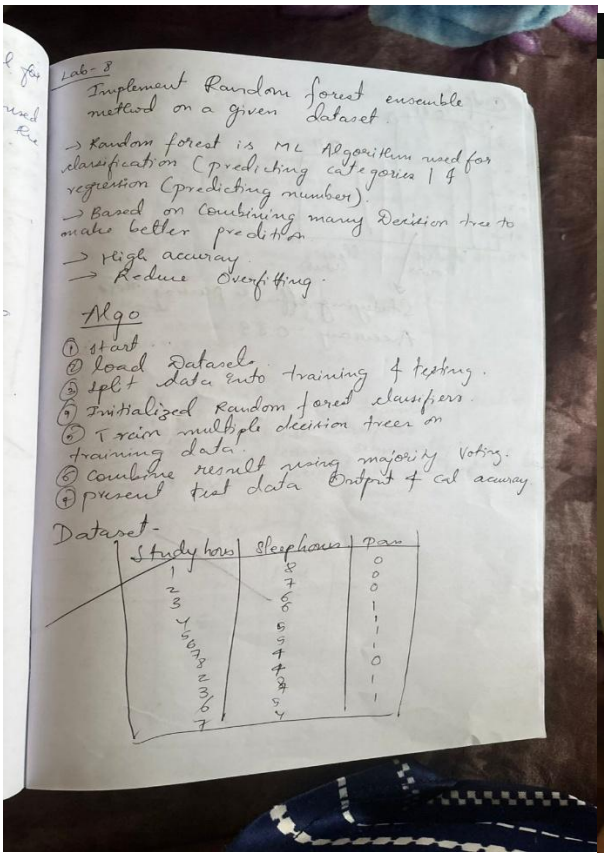
```python
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:\n", cm)

# Classification Report
cr = classification_report(y_test, y_pred)
print("\nClassification Report:\n", cr)
```

```
Accuracy: 0.9666666666666667

Confusion Matrix:
 [[10  0  0]
 [ 0  8  1]
 [ 0  0 11]]

Classification Report:
               precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      0.89      0.94         9
           2       0.92      1.00      0.96        11

    accuracy                           0.97        30
   macro avg       0.97      0.96      0.97        30
weighted avg       0.97      0.97      0.97        30
```

Program 8

Implement Random forest ensemble method on a given dataset.

Screenshot



Lab-8

Implement Random forest ensemble method on a given dataset.

→ Random forest is ML Algorithm used for classification (predicting categories) & regression (predicting number).

→ Based on combining many Decision trees to make better predictions.

→ High accuracy.

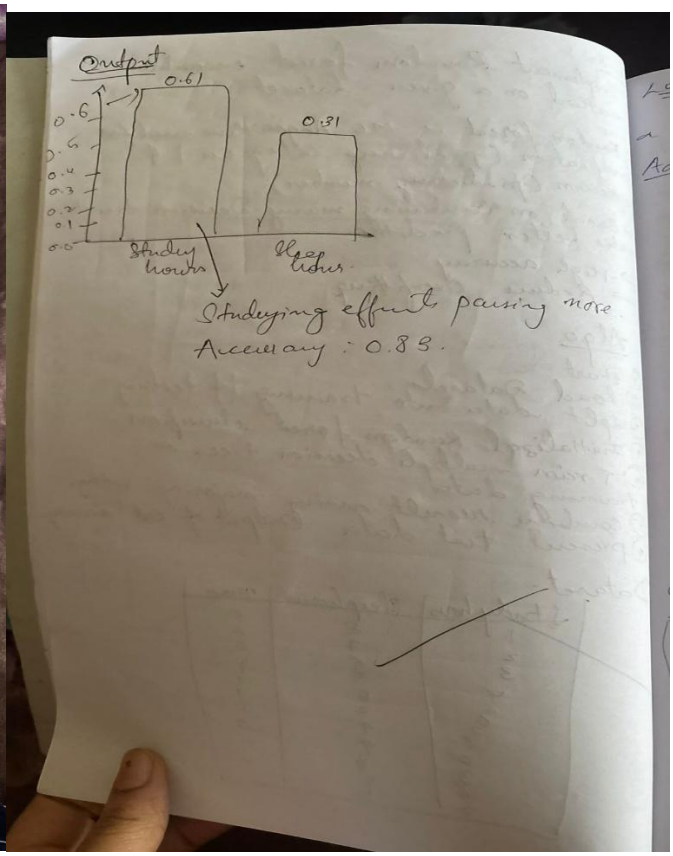
→ Reduce Overfitting.

Algo

1. start
2. load Dataset.
3. split data into training & testing.
4. Initialized Random forest classifiers.
5. Train multiple decision trees on training data.
6. Combine result using majority voting.
7. present test data Output & cal accuracy

Dataset-

Study hours	Sleep hours	pass
1	8	0
2	7	0
3	6	0
4	5	1
5	5	1
6	4	1
7	4	0
8	3	1
2	4	1
3	5	1
6	4	
7		



Output

Studying effort passing more

Accuracy : 0.83.

Code:

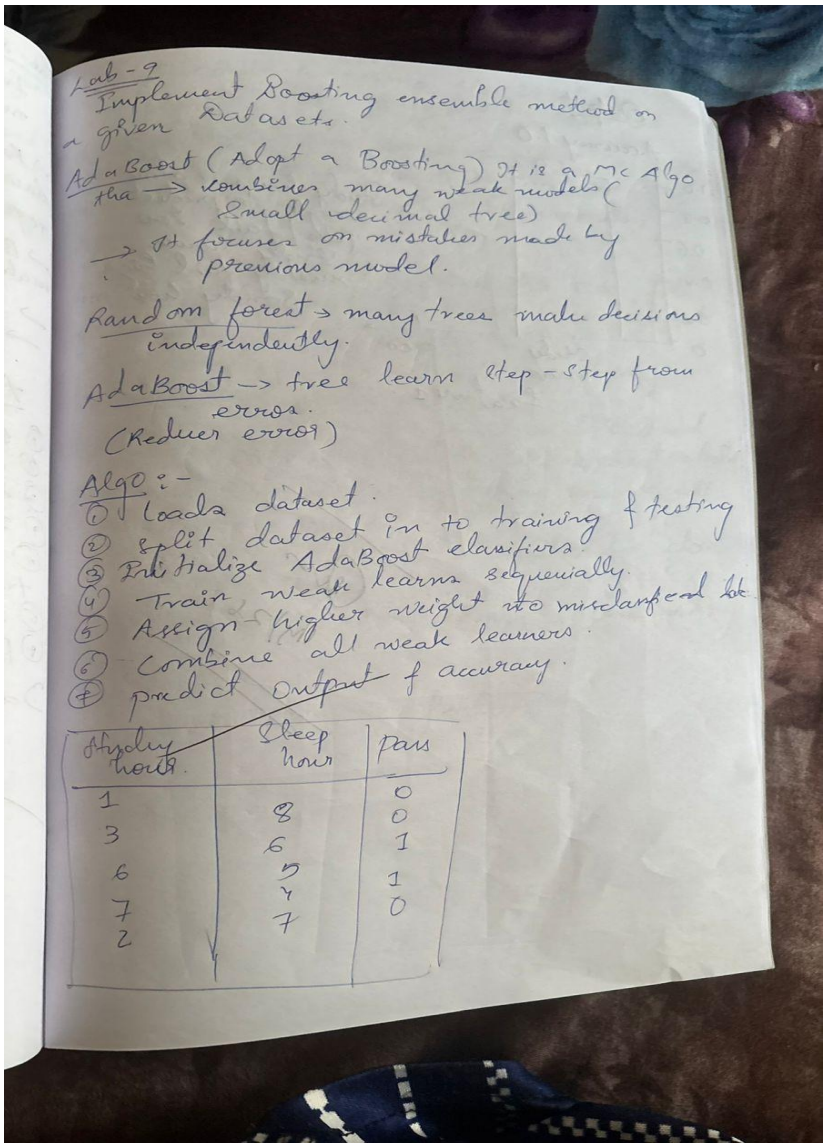
```python
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = RandomForestClassifier(n_estimators=100, random_state=42)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
```

```
Accuracy: 1.0
Confusion Matrix:
 [[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

Program 9

Implement Boosting ensemble method on a given dataset.

Screenshot



Lab - 9

Implement Boosting ensemble method on a given Dataset.

AdaBoost (Adapt a Boosting) It is a ML Algo that → combines many weak models (small decimal tree)

→ It focuses on mistakes made by previous model.

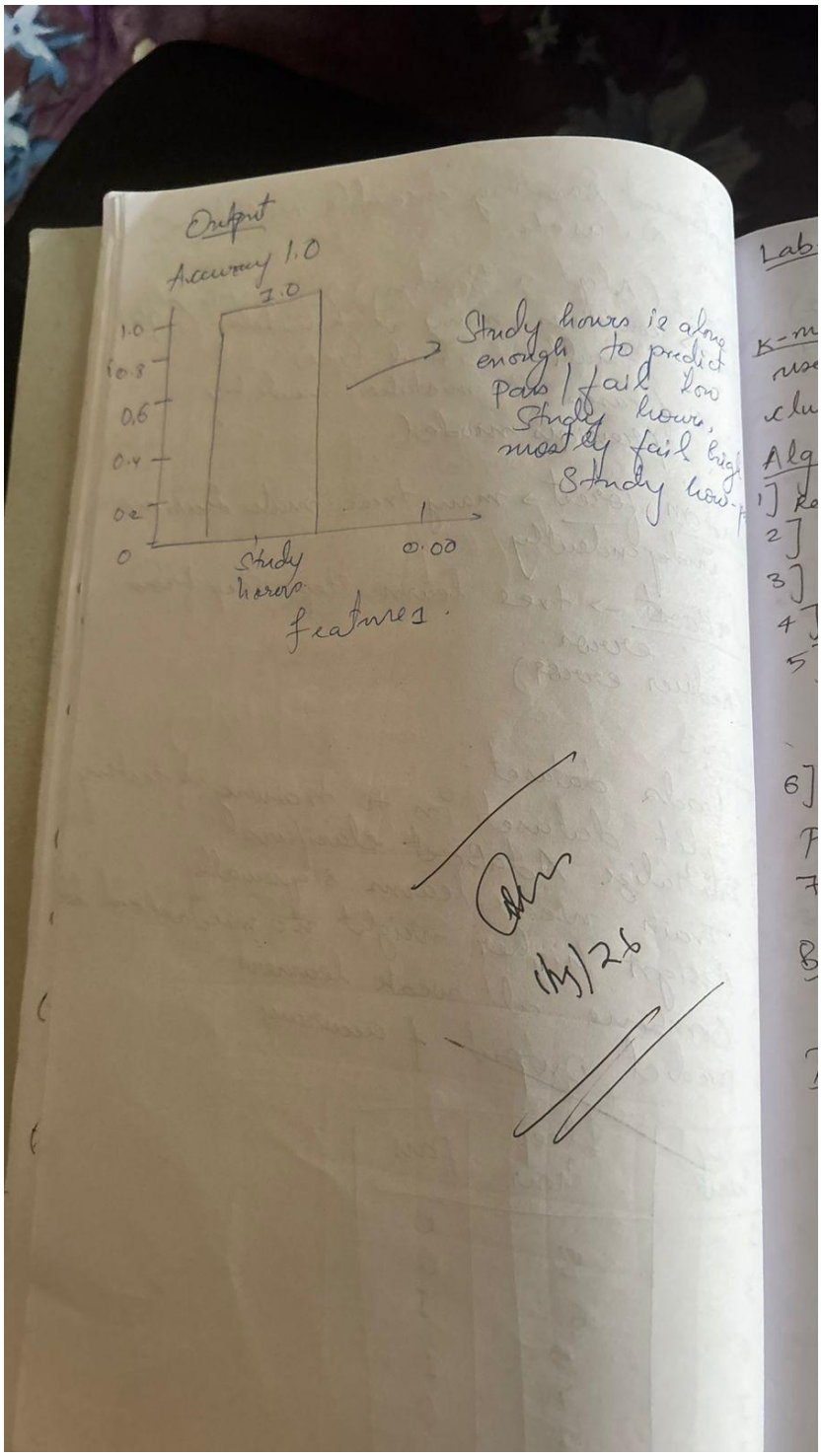
Random forest → many trees make decisions independently.

AdaBoost → tree learn step - step from error.
(Reduces error)

Algo :-

1. Loads dataset.
2. Split dataset in to training & testing
3. Initialize AdaBoost classifiers.
4. Train weak learns sequentially.
5. Assign higher weight to misclassified data.
6. Combine all weak learners.
7. predict Output & accuracy.

Study hours	Sleep hour	pass
1	8	0
3	6	0
6	5	1
7	4	1
2	7	0


Output
Accuracy 1.0
1.0
0.8
0.6
0.4
0.2
0
Study hours
0.00
features
Study hours is alone enough to predict Pass / fail low Study hours, mostly fail high Study hours
(M)/26

Code:

```python
# Import libraries
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load Wine dataset
data = load_wine()
X = data.data
y = data.target

# Split dataset into train and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create AdaBoost model
model = AdaBoostClassifier(
    n_estimators=50,      # number of weak learners
    learning_rate=1.0,    # contribution of each classifier
    random_state=42
)

# Train model
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Evaluate model
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

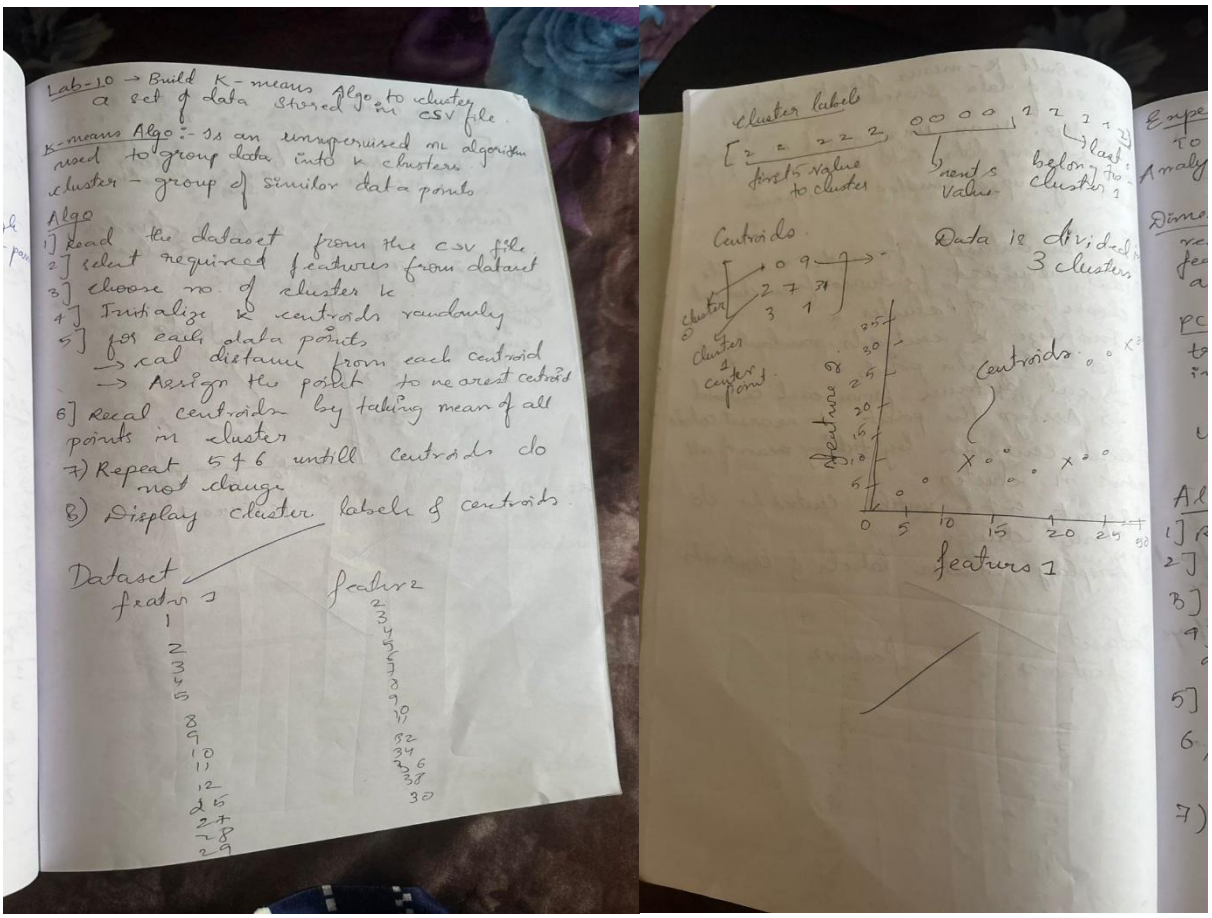
```python
# Evaluate model
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
Accuracy: 0.9444444444444444

Classification Report:
               precision    recall  f1-score   support

           0       0.93      1.00      0.97        14
           1       0.93      0.93      0.93        14
           2       1.00      0.88      0.93         8

    accuracy                           0.94        36
   macro avg       0.95      0.93      0.94        36
weighted avg       0.95      0.94      0.94        36
```

Program 10

Build k-Means algorithm to cluster a set of data stored in a .CSV file.

Screenshot



Code:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
```

```
data = pd.read_csv("/content/data.csv")
print(data.head())
```

```python
X = data.iloc[:, :].values   # or choose specific columns
```

```python
k = 3   # you can change this
```

```python
kmeans = KMeans(n_clusters=k, random_state=0)
kmeans.fit(X)
```

```python
labels = kmeans.labels_
print(labels)
```

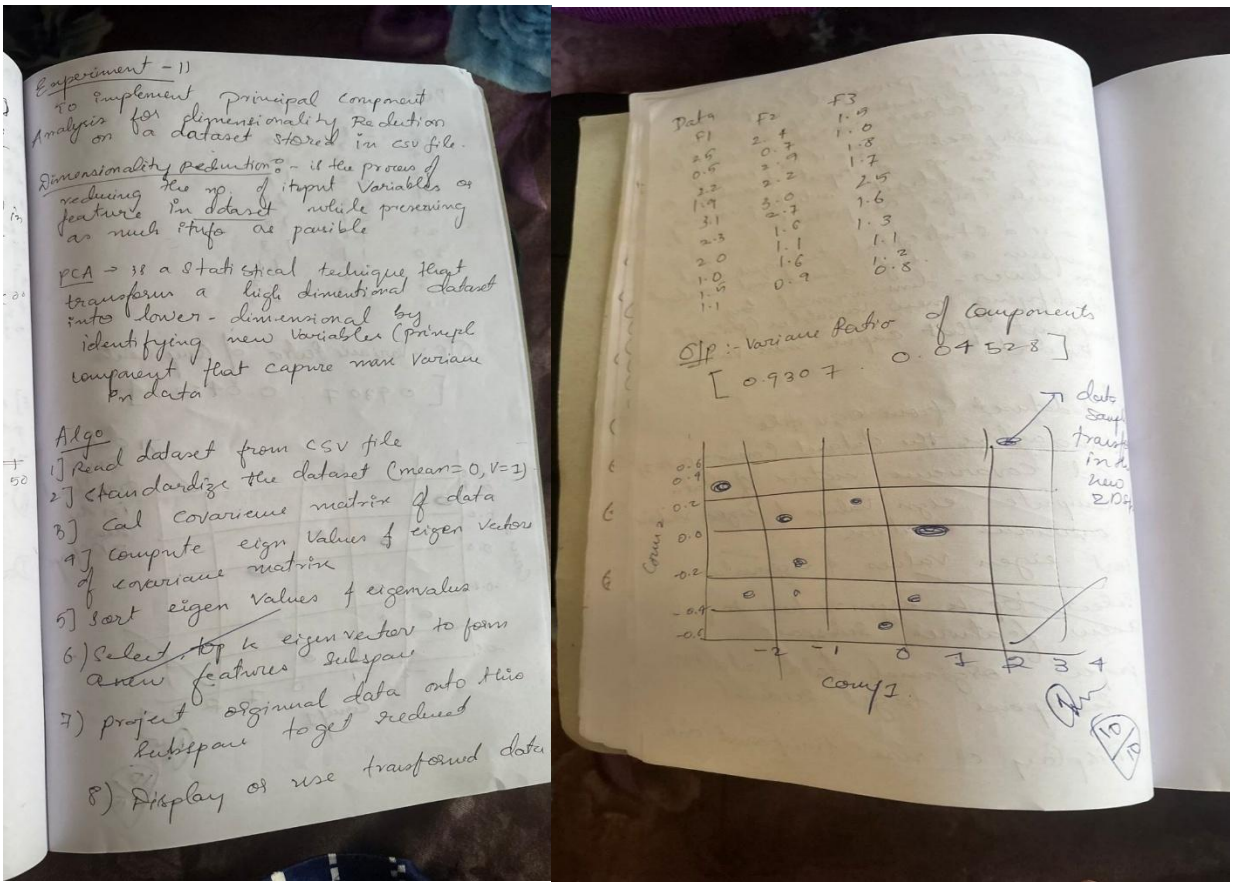
```python
centroids = kmeans.cluster_centers_
print(centroids)
```

```python
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')
plt.scatter(centroids[:, 0], centroids[:, 1], s=200, c='red')
plt.title("K-Means Clustering")
plt.show()
```

Program 11

Implement Dimensionality reduction using Principle Component Analysis (PCA) method.

Screenshot



Experiment - 11

To implement Principal Component Analysis for dimensionality Reduction on a dataset stored in csv file.

Dimensionality reduction :- is the process of reducing the no. of input variables or feature in dataset while preserving as much info as possible

PCA → is a statistical technique that transforms a high dimensional dataset into lower-dimensional by identifying new variables (principal component that capture max variance in data

Algo

1] Read dataset from CSV file
2] standardize the dataset (mean = 0, V = 1)
3] Cal covariance matrix of data
4] compute eign values & eigen vectors of covariance matrix
5] sort eigen values & eigenvalues
6] Select ~~top~~ k eigenvectors to form a new features subspace
7] project original data onto this subspace to get reduced
8] Display or use transformed data

Data F1	F2	F3
2.5	2.4	1.9
0.5	0.7	1.0
2.2	2.9	1.8
1.9	2.2	1.7
3.1	3.0	2.5
2.3	2.7	1.6
2.0	1.6	1.3
1.0	1.1	1.1
1.5	1.6	1.2
1.1	0.9	0.8

O/P :- Variance Ratio of components

[0.9307 . 0.04528]

Code:

PCA

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
```

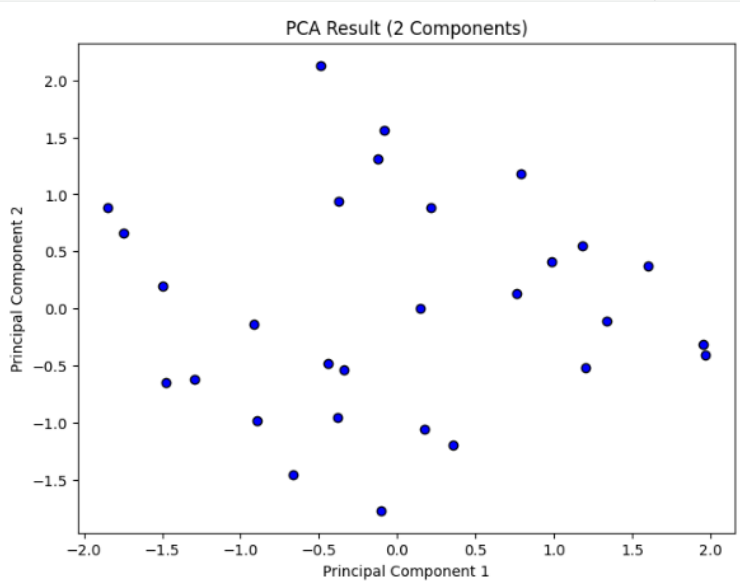
```
# Load dataset
data = pd.read_csv("data.csv")
print(data.head())
```

```
    x   y
0  30  49
1  31  27
2  31  46
3   1  65
4  81  76
```

```
# Standardize the data
scaler = StandardScaler()
scaled_data = scaler.fit_transform(data)
```

```
# Apply PCA
pca = PCA(n_components=2)  # Reduce to 2 components
pca_result = pca.fit_transform(scaled_data)
```

```
# Explained variance ratio
print(f"Explained Variance Ratio: {pca.explained_variance_ratio_}")
print(f"Total Variance Explained by 2 Components: {np.sum(pca.explained_variance_ratio_)}")
```

```
Explained Variance Ratio: [0.56506606 0.43493394]
Total Variance Explained by 2 Components: 1.0
```

```
# Scatter plot of the 2D PCA result
plt.figure(figsize=(8, 6))
plt.scatter(pca_result[:, 0], pca_result[:, 1], c='blue', edgecolors='black')
plt.title('PCA Result (2 Components)')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.show()
```